AfriOS

Guide de build définitif

Kali Linux — du clone au boot QEMU

Toutes les commandes, dans l'ordre, sans détour.
Pré-requis → clone → vendorisation → build (5 couches + 3 noyaux + firmware) → tests → boot QEMU → rapport.

VERSION
1.0

DATE
10 November 2026

AUTEUR
Équipe AfriOS
Document technique AfriOS — github.com/ErdisKodjo/System

STATUT
Public

Table des matières

Table des matières 2
1. Pré-requis système (une seule fois) 4
1.1 Mettre à jour le système . . . 4
1.2 Installer tous les outils en une commande . . . 4
1.3 Vérifier les versions installées . . . 4
2. Cloner le dépôt AfriOS 5
3. Vendoriser les dépendances externes 6
3.1 Vendorisation minimale (recommandé pour démarrer) . . . 6
3.2 Vendorisation complète (avec EDK2 pour le firmware) . . . 6
3.3 Vérifier les dépendances . . . 6
4. Setup one-shot (validation rapide, 8 secondes) 7
5. Build des 5 couches de compatibilité (5 min) 8
5.1 Build module par module . . . 8
5.2 Vérifier les artifacts produits . . . 8
6. Build du noyau (4 architectures) 9
6.1 Build ARM64 (Raspberry Pi 4, Pine64, serveurs) . . . 9
6.2 Build x86_64 (desktops, serveurs — natif) . . . 9
6.3 Build RISC-V 64 (SiFive, QEMU virt) . . . 9
6.4 Build MCU Cortex-M (expérimental) . . . 9
6.5 Vérifier les kernels produits . . . 9
7. Build du firmware UEFI (nécessite EDK2) 11
7.1 Configurer l'environnement EDK2 . . . 11
7.2 Build firmware X64 (QEMU, serveurs x86) . . . 11
7.3 Build firmware AARCH64 (Rpi4, Pine64) . . . 11
7.4 Vérifier les firmware produits . . . 11
8. Lancer les tests 12
8.1 Tests HAL (42 tests, 0.5 seconde) . . . 12
8.2 Vérification syntaxique (334 fichiers, 8 secondes) . . . 12
8.3 Tests de compatibilité Linux . . . 12
8.4 Tous les tests en une commande . . . 12
9. Boot test sur QEMU (validation runtime) 13
9.1 Boot du noyau ARM64 sur QEMU virt . . . 13

9.2 Boot du firmware X64 sur QEMU . . . 13
10. Générer le rapport de build **14**
11. Récapitulatif — tout en une commande **15**
12. En cas d'erreur **16**
12.1 Erreurs CMake . . . 16
12.2 Erreurs cross-compilation . . . 16
12.3 Erreurs MCU . . . 16
12.4 Erreurs QEMU . . . 16
12.5 Erreurs EDK2 . . . 16
13. Récapitulatif des artifacts finaux **17**
13.1 Bibliothèques de compatibilité . . . 17
13.2 Kernels . . . 17
13.3 Firmware UEFI (si EDK2 vendorisé) . . . 17
13.4 Rapports . . . 17
14. Conclusion **18**

1. Pré-requis système (une seule fois)

Avant de cloner AfriOS, installez tous les outils nécessaires. Cette étape ne se fait qu'une fois. Kali Linux étant basé sur Debian Testing, la plupart des paquets sont disponibles via apt.

1.1 Mettre à jour le système

```
sudo apt update && sudo apt upgrade -y
```

1.2 Installer tous les outils en une commande

Copiez-collez cette commande unique dans votre terminal :

```
sudo apt install -y \ build-essential gcc g++ make cmake ninja-build \ python3
python3-pip git curl wget unzip pkg-config libssl-dev \ gcc-aarch64-linux-gnu
gcc-riscv64-linux-gnu gcc-arm-none-eabi \ x86_64-w64-mingw32-gcc gobjc gobjc-14
clang libobjc-dev \ qemu-system-arm qemu-system-x86 qemu-system-misc
qemu-user-static \ nasm acpica-tools
```

1.3 Vérifier les versions installées

```
gcc --version # doit afficher 14.x ou 16.x cmake --version # doit afficher 3.28+
aarch64-linux-gnu-gcc --version # cross-compiler ARM64 riscv64-linux-gnu-gcc
--version # cross-compiler RISC-V arm-none-eabi-gcc --version # cross-compiler MCU
qemu-system-aarch64 --version # émulateur QEMU gcc -x objective-c -fsyntax-only
/dev/null # vérifie gobjc
```

Si la dernière commande n'affiche pas d'erreur, gobjc est bien installé et les fichiers Objective-C (.m) pourront être compilés.

2. Cloner le dépôt AfriOS

```
cd ~ git clone https://github.com/ErdisKodjo/System.git AfriOS cd AfriOS ls -la
```

Le dépôt est petit (~5 MB). La structure doit contenir : AfriOS/ (code source), scripts/ (automatisation), tests/ (tests de compatibilité), ci-workflows/ (GitHub Actions).

3. Vendoriser les dépendances externes

AfriOS dépend de 9 bibliothèques externes (EDK2, Wine, ART, Darling, etc.). Elles ne sont pas commités dans le dépôt (trop volumineuses) mais sont téléchargées via un script reproductible.

3.1 Vendorisation minimale (recommandé pour démarrer)

Pour un premier build sans le firmware UEFI, vous n'avez besoin que de Vulkan-Headers et glslang (pour DXVK). ~80 MB, 1 minute.

```
./scripts/fetch-deps.sh vulkan ./scripts/fetch-deps.sh glslang
```

3.2 Vendorisation complète (avec EDK2 pour le firmware)

Pour builder le firmware UEFI, vendorisez EDK2 (~5 GB, 30-60 min selon votre connexion). Vous pouvez aussi tout télécharger d'un coup.

```
# Option A : EDK2 seul ./scripts/fetch-deps.sh edk2 # Option B : tout en un (~6.5 GB, 60-90 min) ./scripts/fetch-deps.sh all
```

3.3 Vérifier les dépendances

```
./scripts/check-deps.sh # Attendu : 9/9 OK (ou au moins vulkan + glslang + edk2)
```

4. Setup one-shot (validation rapide, 8 secondes)

Avant de lancer les builds complets, exécutez le script setup.sh qui valide en une commande : la présence des outils, la syntaxe de tous les fichiers source, les tests HAL, et la découverte des tests de compatibilité.

```
./scripts/setup.sh --all --skip fetch-edk2
```

Résultat attendu :

```
Phase Result Elapsed Critical ---------- ---------- ---------- ---------- check-deps pass 0s 1 install-workflows pass 0s 1 fetch-edk2 skip 0s 0 syntax-check pass 8s 1 hal-tests pass 0s 1 compat-tests pass 0s 1 Critical failures: 0 [ok] setup complete — every critical phase passed.
```

Si cette étape passe, vous êtes prêt pour les builds complets. Si elle échoue, corrigez d'abord le problème avant de continuer.

5. Build des 5 couches de compatibilité (5 min)

Cette étape build les 6 modules userspace (5 couches de compat + l'orchestrateur CoreBridge). Tous produisent des bibliothèques partagées (.so) ou statiques (.a). Le build se fait en natif x86_64 (pas de cross-compilation).

5.1 Build module par module

```
cd ~/AfriOS/AfriOS/AfriOS/OS for module in afros-winbridge afros-androsandbox afros-harmonygate \ afros-dxvk afros-corebridge-core afros-incompat-engine; do echo "=== Building $module ===" cd $module cmake -B build -S . && cmake --build build -j$(nproc) ls -la build/*.so build/*.a 2>/dev/null cd .. done
```

5.2 Vérifier les artifacts produits

Vous devez obtenir ces 6 bibliothèques :

Module	Fichier	Taille attendue
afros-winbridge	build/libafros-winbridge.so	~104 KB
afros-androsandbox	build/libafros-androsandbox.so	~1.5 MB
afros-harmonygate	build/libafros-harmonygate.so	~390 KB
afros-dxvk	build/libafros-dxvk.so + 4 symlinks DLL	~430 KB
afros-corebridge-core	build/libafros-corebridge.a	~148 KB
afros-incompat-engine	build/libafros-incompat-engine.so	~260 KB

Revenez ensuite à la racine du dépôt pour les builds noyau :

```
cd ~/AfriOS
```

6. Build du noyau (4 architectures)

Le noyau AfriOS est multi-architectures. Chaque build cible une architecture via la variable AFROS_PORT. En cross-compilation (ARM64, RISC-V, MCU), seuls le noyau + HAL + drivers + port sont construits — les modules userspace sont skippés (ils sont déjà construits en natif à l'étape 5).

6.1 Build ARM64 (Raspberry Pi 4, Pine64, serveurs)

```
cd ~/AfriOS/AfriOS/AfriOS/OS cmake -B build-arm64 -S . \ -DAFROS_PORT=arm64 \
-DAFROS_BUILD_KERNEL=ON \ -DAFROS_BUILD_TESTS=OFF \
-DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/toolchain-arm64.cmake cmake --build
build-arm64 -j$(nproc)
```

6.2 Build x86_64 (desktops, serveurs — natif)

Le build x86_64 est natif (pas de cross-compilation). Il construit le noyau ET tous les modules userspace.

```
cmake -B build-x86_64 -S . \ -DAFROS_PORT=x86_64 \ -DAFROS_BUILD_KERNEL=ON \
-DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/toolchain-x86_64.cmake cmake --build
build-x86_64 -j$(nproc)
```

6.3 Build RISC-V 64 (SiFive, QEMU virt)

```
cmake -B build-riscv -S . \ -DAFROS_PORT=riscv \ -DAFROS_BUILD_KERNEL=ON \
-DAFROS_BUILD_TESTS=OFF \
-DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/toolchain-riscv.cmake cmake --build
build-riscv -j$(nproc)
```

6.4 Build MCU Cortex-M (expérimental)

Le port MCU est expérimental (bare-metal freestanding). Il peut échouer — c'est normal. Ajoutez || true pour ne pas bloquer le script.

```
cmake -B build-mcu -S . \ -DAFROS_PORT=mcu \ -DAFROS_BUILD_KERNEL=ON \
-DAFROS_BUILD_TESTS=OFF \
-DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/toolchain-mcu.cmake cmake --build
build-mcu -j$(nproc) || echo "MCU: experimental, skip"
```

6.5 Vérifier les kernels produits

```
cd ~/AfriOS/AfriOS/AfriOS/OS echo "=== Kernel ELF files ===" find build-arm64
build-x86_64 build-riscv \ -name "afros-kernel-sim" -type f | xargs ls -lh
2>/dev/null echo "=== MCU static lib ===" find build-mcu -name "libafros-kernel.a"
| xargs ls -lh 2>/dev/null
```

Résultat attendu :

```
=== Kernel ELF files === -rwxrwxr-x 1 erdis erdis 1.5M
build-arm64/afros-core/Kernel/afros/afros-kernel-sim -rwxrwxr-x 1 erdis erdis 1.8M
build-x86_64/afros-core/Kernel/afros/afros-kernel-sim -rwxrwxr-x 1 erdis erdis 1.4M
build-riscv/afros-core/Kernel/afros/afros-kernel-sim === MCU static lib === -rw-rw-r-- 1 erdis
```

```
erdis 280K build-mcu/afros-core/Kernel/afros/libafros-kernel.a
```

Revenez à la racine :

```
cd ~/AfriOS
```

7. Build du firmware UEFI (nécessite EDK2)

Cette étape nécessite qu'EDK2 soit vendorisé (étape 3.2). Si vous n'avez pas téléchargé EDK2, sautez cette section et passez à l'étape 8 (tests).

7.1 Configurer l'environnement EDK2

```
cd ~/AfriOS/AfriOS/FirmwareHybride/edk2 source edksetup.sh export WORKSPACE=$(pwd)
export PACKAGES_PATH=$WORKSPACE:$WORKSPACE/HybridFirmwarePlatformPkg
```

7.2 Build firmware X64 (QEMU, serveurs x86)

```
build -p HybridFirmwarePlatformPkg/HybridFirmwarePlatformPkg.dsc \ -a X64 -t GCC5
-b RELEASE
```

7.3 Build firmware AARCH64 (Rpi4, Pine64)

```
build -p HybridFirmwarePlatformPkg/HybridFirmwarePlatformPkg.dsc \ -a AARCH64 -t
GCC5 -b RELEASE
```

7.4 Vérifier les firmware produits

```
ls                                                                          -lh
Build/HybridFirmwarePlatformPkg/RELEASE_GCC5/FV/HYBRIDFIRMWAREPLATFORMPKG.FD     #
Attendu : ~4 MB pour X64, ~5 MB pour AARCH64
```

Revenez à la racine :

```
cd ~/AfriOS
```

8. Lancer les tests

8.1 Tests HAL (42 tests, 0.5 seconde)

```
./scripts/run-hal-tests.sh # Attendu : 42 PASS / 0 FAIL / 2 SKIP
```

8.2 Vérification syntaxique (334 fichiers, 8 secondes)

```
python3 scripts/ci-syntax-check.py # Attendu : 0 failed
```

8.3 Tests de compatibilité Linux

```
cd tests/linux make -C hello-elf && ./hello-elf/hello-elf make -C fork-exec &&
./fork-exec/fork-exec cd ~/AfriOS
```

Résultat attendu : Hello, AfriOS!

8.4 Tous les tests en une commande

```
./scripts/run-all-tests.sh # Génère : tests/results/full-report-<timestamp>.md
```

9. Boot test sur QEMU (validation runtime)

Cette étape est la validation ultime : booter le noyau AfriOS cross-compilé sur QEMU pour vérifier qu'il s'exécute.

9.1 Boot du noyau ARM64 sur QEMU virt

```
qemu-system-aarch64 \ -machine virt \ -cpu cortex-a72 \ -m 2G \ -kernel ~/AfriOS/AfriOS/AfriOS/OS/build-arm64/afros-core/Kernel/afros/afros-kernel-sim \ -nographic \ -no-reboot
```

Sortie attendue :

> --- Bienvenue dans AfriOS Core (ARM64 v8.5) --- [KERNEL] Démarrage du système de démarrage... [HAL] Console initialized [HAL] CPU initialized [HAL] Memory initialized [HAL] Interrupts initialized [HAL] Timer initialized [HAL] Storage initialized [POWER] Source Batterie/AC. Mode économie d'énergie. [KERNEL] Noyau opérationnel. Lancement du planificateur... [KERNEL] Arrêt du système ou Idle...

Pour quitter QEMU : Ctrl+A puis X

9.2 Boot du firmware X64 sur QEMU

Si vous avez buildé le firmware UEFI (étape 7), vous pouvez le booter sur QEMU :

```
qemu-system-x86_64 \ -m 2G \ -bios ~/AfriOS/AfriOS/FirmwareHybride/edk2/Build/HybridFirmwarePlatformPkg/RELEASE_GCC5/FV/HYBRIDFIRMWAREPLATFORMPKG.FD \ -nographic
```

Sortie attendue :

> UEFI firmware AfriOS (HybridFirmwarePlatformPkg) v1.0 Press ESC for boot menu... Starting AfriOS kernel...

10. Générer le rapport de build

```
python3 ~/AfriOS/scripts/generate-build-report.py cat ~/AfriOS/build/reports/build-report-latest.md
```

Le rapport contient : la liste de tous les artifacts produits (.so, .a, .elf, .fd) avec leur taille et leur hash SHA-256, le récapitulatif des tests HAL, et la liste des modules built/skipped.

11. Récapitulatif — tout en une commande

Si vous avez déjà installé les pré-requis (étape 1), voici la séquence complète en un seul bloc copiable :

```
# 1. Cloner cd ~ git clone https://github.com/ErdisKodjo/System.git AfriOS cd AfriOS # 2. Vendoriser les deps minimales ./scripts/fetch-deps.sh vulkan ./scripts/fetch-deps.sh glslang # 3. Validation rapide (8s) ./scripts/setup.sh --all --skip fetch-edk2 # 4. Build des 5 couches de compat cd AfriOS/AfriOS/OS for m in afros-winbridge afros-androsandbox afros-harmonygate \ afros-dxvk afros-corebridge-core afros-incompat-engine; do cd $m && cmake -B build -S . && cmake --build build -j$(nproc) && cd .. done # 5. Build des noyaux (3 archs) cmake -B build-arm64 -S . -DAFROS_PORT=arm64 -DAFROS_BUILD_KERNEL=ON \
-DAFROS_BUILD_TESTS=OFF
-DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/toolchain-arm64.cmake cmake --build build-arm64 -j$(nproc) cmake -B build-x86_64 -S . -DAFROS_PORT=x86_64 -DAFROS_BUILD_KERNEL=ON \
-DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/toolchain-x86_64.cmake cmake --build build-x86_64 -j$(nproc) cmake -B build-riscv -S . -DAFROS_PORT=riscv -DAFROS_BUILD_KERNEL=ON \ -DAFROS_BUILD_TESTS=OFF
-DCMAKE_TOOLCHAIN_FILE=cmake/toolchains/toolchain-riscv.cmake cmake --build build-riscv -j$(nproc) # 6. Tests cd ~/AfriOS ./scripts/run-all-tests.sh # 7. Boot test QEMU qemu-system-aarch64 -machine virt -cpu cortex-a72 -m 2G \ -kernel ~/AfriOS/AfriOS/AfriOS/OS/build-arm64/afros-core/Kernel/afros/afros-kernel-sim \
-nographic -no-reboot # 8. Rapport python3 ~/AfriOS/scripts/generate-build-report.py cat ~/AfriOS/build/reports/build-report-latest.md
```

12. En cas d'erreur

12.1 Erreurs CMake

· « Could not find toolchain file » — vous n'êtes pas dans le bon répertoire. Vous devez être dans ~/AfriOS/AfriOS/AfriOS/OS/ avant de lancer cmake.

· « CMAKE_C_COMPILER not set » — le toolchain file est introuvable. Vérifiez le chemin : cmake/toolchains/toolchain-arm64.cmake.

12.2 Erreurs cross-compilation

· « cannot find -lafros-port-host-mock » — les tests HAL ne peuvent pas être construits en cross-compilation. Ajoutez -DAFROS_BUILD_TESTS=OFF à votre commande cmake.

· « EM: 183 — fichier dans un mauvais format » — les modules userspace ne peuvent pas être cross-compilés. C'est normal : en cross-compile, seul le noyau est construit. Les modules userspace sont construits séparément en natif.

12.3 Erreurs MCU

· « unknown cpu cortex-m » — utilisez cortex-m4 ou cortex-m3 dans le fichier vector_table.S.

· « pthread.h: No such file or directory » — le port MCU est freestanding (pas de libc). Les modules userspace sont skippés automatiquement en mode freestanding.

12.4 Erreurs QEMU

· « kvm: disabled by host » — KVM n'est pas disponible. QEMU fonctionnera en mode TCG (émulation logicielle, plus lent mais fonctionnel). Retirez -enable-kvm de la commande.

· Pour quitter QEMU : appuyez sur Ctrl+A puis X.

12.5 Erreurs EDK2

· « build: command not found » — vous n'avez pas sourc edksetup.sh. Lancez : source edksetup.sh depuis le répertoire external/edk2/.

· « could not find BaseTools » — les submodules EDK2 ne sont pas initialisés. Lancez : git -C external/edk2 submodule update --init --depth 1 puis make -C external/edk2/BaseTools/Source/C.

13. Récapitulatif des artifacts finaux

À la fin du build complet, vous devez avoir :

13.1 Bibliothèques de compatibilité

~/AfriOS/AfriOS/AfriOS/OS/afros-winbridge/build/libafros-winbridge.so
~/AfriOS/AfriOS/AfriOS/OS/afros-androsandbox/build/libafros-androsandbox.so
~/AfriOS/AfriOS/AfriOS/OS/afros-harmonygate/build/libafros-harmonygate.so
~/AfriOS/AfriOS/AfriOS/OS/afros-dxvk/build/libafros-dxvk.so + d3d9.dll.so, d3d11.dll.so, d3d12.dll.so, dxgi.dll.so (symlinks)
~/AfriOS/AfriOS/AfriOS/OS/afros-corebridge-core/build/libafros-corebridge.a
~/AfriOS/AfriOS/AfriOS/OS/afros-incompat-engine/build/libafros-incompat-engine.so

13.2 Kernels

~/AfriOS/AfriOS/AfriOS/OS/build-arm64/afros-core/Kernel/afros/afros-kernel-sim (ARM64 ELF, ~1.5 MB, bootable sur QEMU virt)
~/AfriOS/AfriOS/AfriOS/OS/build-x86_64/afros-core/Kernel/afros/afros-kernel-sim (x86_64 ELF, ~1.8 MB, bootable sur QEMU x86_64)
~/AfriOS/AfriOS/AfriOS/OS/build-riscv/afros-core/Kernel/afros/afros-kernel-sim (RISC-V ELF, ~1.4 MB, bootable sur QEMU virt RISC-V)

13.3 Firmware UEFI (si EDK2 vendorisé)

~/AfriOS/AfriOS/FirmwareHybride/edk2/Build/HybridFirmwarePlatformPkg/RELEASE_GCC5/FV/HYBRIDFIRMWAREPLATFORMPKG.FD (X64, ~4 MB)
HYBRIDFIRMWAREPLATFORMPKG.FD (AARCH64, ~5 MB)

13.4 Rapports

~/AfriOS/build/reports/build-report-latest.md (résumé Markdown)
~/AfriOS/build/reports/build-manifest-latest.json (hashes SHA-256)
~/AfriOS/tests/results/hal-tests-*.log (42 tests HAL) ~/AfriOS/tests/results/full-report-*.md (tous les tests)

14. Conclusion

Ce guide vous a conduit pas-à-pas, depuis l'installation des pré-requis jusqu'au boot test sur QEMU. À la fin, vous disposez d'un build AfriOS complet : 6 bibliothèques de compatibilité, 3 kernels cross-compilés (ARM64, x86_64, RISC-V), et optionnellement 2 firmware UEFI (X64, AARCH64).

Le build complet (sans EDK2) prend environ 15 minutes sur une machine moderne. Avec EDK2, comptez 60-90 minutes supplémentaires pour le téléchargement et 30 minutes pour le build firmware.

Pour toute question ou problème, consultez le worklog consolidé WORKLOG-COMPLETION.md (3 100+ lignes) qui documente toutes les décisions techniques, ou ouvrez une issue sur GitHub : https://github.com/ErdisKodjo/System/issues